1. Hãy vẽ lưu đồ thuật toán của các thuật toán (khuyên dùng draw.io)

Insertionsort.drawio.pdf

Quicksort.drawio.pdf

Selectionsort.drawio.pdf

2. Quan sát đường biểu diễn của Selection Sort trên cả 4 bài đo (ngẫu nhiên, gần như sắp xếp, đảo ngược, trùng lặp). Thời gian chạy có thay đổi đáng kể không? Vì sao?

- Thời gian chạy của Selection Sort thay đổi rất nhiều và biến động mạnh trong cả 4 bài đo của em.
- Theo em, sự biến động lớn trên đồ thị đến nhiều từ việc có thể hoán đổi phần từ ngay cả khi không cần thiết, việc ghi lại phần tử ở mỗi bước hoán đổi làm thời gian dao động mạnh

3. Khi nào Insertion Sort vươn lên vị trí dẫn đầu? Gợi ý: nhìn vào cấu trúc vòng lặp của bạn, số phép hoán đổi thay đổi như thế nào khi mảng đã gần sắp xếp hoặc có nhiều giá trị bằng nhau?

- Insertion Sort vươn lên dẫn đầu ở kịch bản Nearly Sorted và Few Unique
- Vòng lặp bên trong Insertion Sort dừng ngay khi gặp một phần tử nhỏ hơn hoặc bằng phần tử đang xét. Do đó, ở 2 kịch bản trên, các phần tử đã ở gần đúng vị trí nên phép dịch chuyển và so sánh trong vòng lặp giảm, dẫn đến độ phức tạp giảm.

4. Tại sao trên dữ liệu ngẫu nhiên ở quy mô mảng lớn (N=10,000→20,000), Quick Sort lại bỏ xa hai thuật toán còn lại?

- Trên quy mô mảng lớn, Quick Sort hoạt động trên hàm đệ quy nên độ phức tạp giảm so với những cách sắp xếp khác, nhất là khi số lượng dữ liệu ở quy mô mảng nhiều.

5. Tại sao ở mảng kích thước nhỏ (N≤100), Quick Sort lại chậm hơn Insertion Sort? Gợi ý: hãy suy nghĩ về hàm đệ quy các hoạt động của hàm đề quy dưới code

- Quick Sort là thuật toán đệ quy, ở mỗi bước phân chia, hệ thống chịu thêm chi phí để quản lý đệ quy. Với mảng nhỏ, số lượng phần tử quá ít khiến chi phí "quản lý" đệ quy lớn hơn so với thời gian thực sự cần thiết để sắp xếp chúng

6. Lập bảng so sánh tổng kết về 3 thuật toán: Độ phức tạp thời gian (Best, Average, Worst Case), dữ liệu nào tối ưu nhất, dữ liệu nào tồi tệ nhất của mỗi thuật toán.

| Thuật toán | Best Case | Average Case | Worst Case | Dữ liệu tối ưu nhất | Dữ liệu tồi tệ nhất |
|---|---|---|---|---|---|
| Selection Sort | $O(N^2)$ | $O(N^2)$ | $O(N^2)$ | Dữ liệu ngẫu nhiên/nhỏ | Mảng có số phép hoán đổi tốn chi phí |
| Insertion Sort | O(N) | $O(N^2)$ | $O(N^2)$ | Gần như đã sắp xếp, nhiều phần tử trùng lặp | Mảng bị đảo ngược hoàn toàn |
| Quick Sort | O(NlogN) | O(NlogN) | $O(N^2)$ | Ngẫu nhiên, phân bố đều | Đã sắp xếp/ Ngược |

7. Bonus: Nhìn vào đường nét đứt QUICK (Ref) của CLB: tại sao phiên bản này không hề bị suy biến ở các trường hợp dữ liệu không tối ưu mà vẫn giữ được tốc độ ổn định?

- Phiên bản này duy trì thời gian phẳng và tối ưu trên mọi kịch bản dữ liệu nhờ 2 kỹ thuật:

  + Chọn Pivot: thay vì luôn chọn phần tử đầu hoặc cuối làm pivot thì ref chọn pivot ngẫu nhiên hoặc lấy trung vị 3 phần từ nên luôn đảm bảo mảng được chia thành 2 nửa cân bằng (NlogN)

  + Phân chia 3 đường: khi đối mặt với dữ liệu trùng lặp nhiều, thuật toán phân chia mảng thành 3 vùng: < pivot, = pivot và > pivot. Các phần từ bằng pivot sẽ bị bỏ qua ở các lượt đệ quy sau nên thuật toán không bị suy biến